



INTERPRETADOR VISUAL DE C99

IFSCee.org

Código C que você *pode ver*

C code you can see

Tornando visível a memória que todo programa C esconde.

Projeto Integrador II · Tecnologia em Análise e Desenvolvimento de Sistemas — IFSC

Gelasio Ebel Junior · gelasioebel.github.io/ifscee.org



O que veremos hoje.



Contexto & problema

Por que aprender C é difícil

01



A solução IFSCee

Tornar o invisível visível

02



Arquitetura & tecnologia

Pipeline e stack web

03



A ferramenta na prática

Interface, memória e execução

04



Engenharia & deploy

Boas práticas e automação

05



Resultados & futuro

Entregas e próximos passos

06

O problema: a memória é invisível.

Em C, tudo o que mais confunde quem aprende acontece num lugar que ninguém vê — a memória. O compilador esconde endereços, frames de pilha e alocações dinâmicas. O aluno escreve, executa e só vê o resultado final.



`código entra → resultado sai`

×

Ponteiros abstratos

“é o endereço de...” sem nunca ver o endereço

×

Stack × Heap

onde cada variável realmente vive?

×

Bugs silenciosos

dangling pointer e undefined behavior

E se o aluno pudesse *ver* a memória trabalhando, passo a passo?

O IFSCee abre a caixa-preta. Cada linha executada mostra, lado a lado, o código-fonte, a árvore sintática, a memória RAM (stack, heap e dados) e a saída do programa.



O que é o IFSCee?

Um **interpretador visual de C99** que executa código passo a passo direto no navegador — sem instalação, sem cadastro, 100% local.

Padrão alvo

C99 (subset didático)



Memória explícita

Stack, heap e dados com endereço, tipo e tamanho visíveis.



AST navegável

A árvore sintática construída antes de rodar uma linha.



Passo a passo

Avance, retroceda ou rode automático em 1x–10x.



27 exemplos prontos

Do printf básico a bitwise, typedef e structs.

Três perfis de usuário.



Estudantes

Aprendendo C pela primeira vez e que precisam de um modelo mental claro de memória.



Professores & monitores

Que querem usar o IFSCee em aulas e exercícios como recurso visual.



Devs experientes

Que precisam explicar conceitos de baixo nível com visualizações concretas.

O que o projeto se propôs a fazer.



Separar responsabilidades Interface, estilo, processamento, documentação e automação isolados.



Facilitar a manutenção Baixo acoplamento e arquivos em locais previsíveis.



Apoiar o aprendizado Documentação detalhada, exemplos e testes.



Publicar automaticamente Workflow de deploy com atualização consistente.



Preservar o histórico Versões paralelas e registro de decisões e melhorias.

O que o interpretador entende.

Um subset de C99 escolhido para cobrir os conceitos que mais importam no aprendizado de memória:

✓ Tipos: int, float, char, ponteiros

✓ Funções, parâmetros e retorno

✓ Stack frames por chamada

✓ malloc / calloc / realloc / free

✓ Structs e typedef

✓ Arrays e strings literais

✓ printf / puts (saída simulada)

✓ Operadores aritméticos e bitwise

Tecnologias — web pura, sem backend.



71%

HTML5

Estrutura da interface



27%

JavaScript ES6

Lexer, parser e interpretador



1,4%

CSS3

Estilização e composição



GitHub Actions + Pages

Build e publicação automatizados



100% client-side

O interpretador roda inteiro no navegador

Pipeline em camadas.



Cada camada tem uma responsabilidade única — o mesmo padrão de sistemas que processam linguagem em etapas.

Da página ao resultado, em 8 passos.

Usuário acessa a página

Interface é carregada

Estilos são aplicados

Entrada é preparada

5 Análise léxica gera tokens

6 Parser organiza a estrutura

7 Interpretador executa a lógica

8 UI mostra o resultado

Anatomia: três regiões, onze zonas.

1

Barra superior

Ação principal, controles passo-a-passo, velocidade 1x–10x e tema.

2

Painel esquerdo

Lista de arquivos salvos no navegador e o editor de código com destaque de sintaxe.

3

Painel direito

Memória RAM, variáveis observadas e a saída do programa.



Fig. layout do IFSCee

Edição × Execução.

Modo de Edição

- Escreve, cola e altera o código C
- Editor totalmente liberado
- Painéis da direita vazios
- Estado padrão ao abrir

Modo de Execução

- Código fica somente-leitura
- Memória reflete a máquina virtual
- Controles passo-a-passo ativos
- Avança, retrocede e roda automático

Navegando no tempo da execução.



Início



Anterior



Próximo



Continuar



Fim



Voltar no tempo é o diferencial

O botão ◀ desfaz cada passo perfeitamente — o aluno revê o estado anterior da memória e entende causa e efeito.



Velocidade 1x – 10x

No modo automático (▶), um slider ajusta o ritmo da execução.

Editor e biblioteca de exemplos.

```
#include <stdio.h>

int main() {
    int age = 20;
    printf("Idade: %d", age);
    return 0;
}
```

Editor com destaque de sintaxe C. A barra do editor permite salvar, abrir exemplos e exportar a AST como JSON.

27

exemplos prontos

Hello World, funções, malloc, structs, ponteiros, bitwise, typedef...

Preparar, passar, voltar.

Ao clicar em Preparar Execução, o IFSCee analisa o código, constrói a AST e entra no modo passo a passo. Cada instrução atualiza memória e saída em tempo real.



Executa uma instrução por vez



Retrocede e desfaz o estado



Memória e saída ao vivo

```
> passo 1: int age  
stack ← age = 20
```

```
> passo 2: printf  
rodata ← "Idade..."
```

```
> saída:  
Idade: 20
```

Memória RAM: *onde tudo fica visível.*

O painel mostra, para cada variável, onde está, quanto ocupa, de que tipo é e qual o endereço — informação que normalmente só um debugger revela.

1	2	3	4	5	6
Nome	Valor	Segmento	Tipo	Tamanho	Endereço
age	20	STACK	int	4 B	0x1000

Cada cartão de variável carrega seis informações — o aluno lê a memória como lê uma tabela.

Stack, Heap e Data, lado a lado.

STACK

Variáveis locais e parâmetros

Crescem e diminuem com as chamadas de função

início **0x1000**

HEAP

Alocação dinâmica

malloc / calloc / realloc — persiste até o free()

início **0x1388**

RODATA

Dados somente-leitura

Strings literais e constantes, byte a byte

início —

Structs e arrays expandidos.



Structs viram sub-tabela

Cada campo aparece individualmente, com nome, tipo e valor. O endereço de um campo é o da struct mais o offset.

Campos da struct · Ponto p1

```
.x  int  → 3  
.y  int  → 7
```



Arrays e strings indexados

Cada posição ganha uma célula [0], [1], [2]... mostrando valor e caractere correspondente.

[0]	[1]	[2]	[3]	[4]	[5]
a	r	r	[	0	]

Variáveis observadas e saída.

Variáveis observadas

Uma lista personalizada do que importa naquele momento. Clique-direito sobre qualquer variável para acompanhá-la enquanto o programa roda.

```
 age = 20  
 arr[0] = 5
```

Saída do programa

Um terminal simulado reúne tudo que o programa imprime via printf e puts.

```
$ ./programa  
Idade: 20  
arr[0] = 5
```

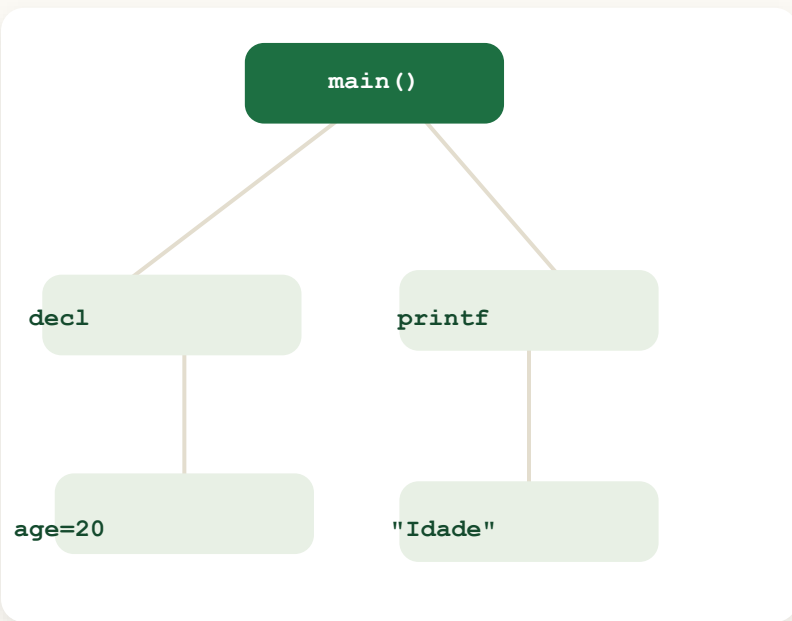
Árvore Sintática Abstrata navegável.

Antes de executar uma única linha, o IFSCee mostra a AST — a estrutura que o parser construiu a partir do código. Dá para exportá-la como JSON.



Exportar AST como JSON

Útil para depurar o parser, comparar estruturas ou estudar como o código vira árvore.



Tema, acessibilidade e UX.



Tema claro / escuro

Alterna a interface inteira com um clique no ícone 🌙.



Sem instalação

Roda em qualquer navegador moderno; internet só para carregar.



Arquivos no navegador

Trabalhos salvos localmente, sem cadastro nem nuvem.



Resposta imediata

Memória e saída atualizam em tempo real a cada passo.

DEMONSTRAÇÃO

Quatro roteiros, do simples ao avançado.

T1

Hello World

Primeiro contato: editar, preparar e avançar.

T2

Funções & Stack

Cada chamada cria um frame novo na pilha.

T3

malloc no Heap

O ponteiro nasce na stack e aponta para o heap.

T4

Structs

Cópia por valor: dois cartões de 8 bytes na stack.

Conceitos que ficam concretos.



Ponteiro = endereço

arr = 5000 mostra, em número, para onde ele aponta.



Por valor × referência

Copiar struct grande duplica bytes — daí passar ponteiro.



Dangling pointer

Após o free(), o ponteiro aponta para memória que não é mais sua.



Undefined behavior

Acessar arr[0] depois do free é sinalizado: aprender pelo erro.

Organização e boas práticas.



Separação de camadas `interface · estilo · lógica · docs ·`
`automação`



Documentação completa `README, manual, IFSCEE.md e pasta`
`documentation/`



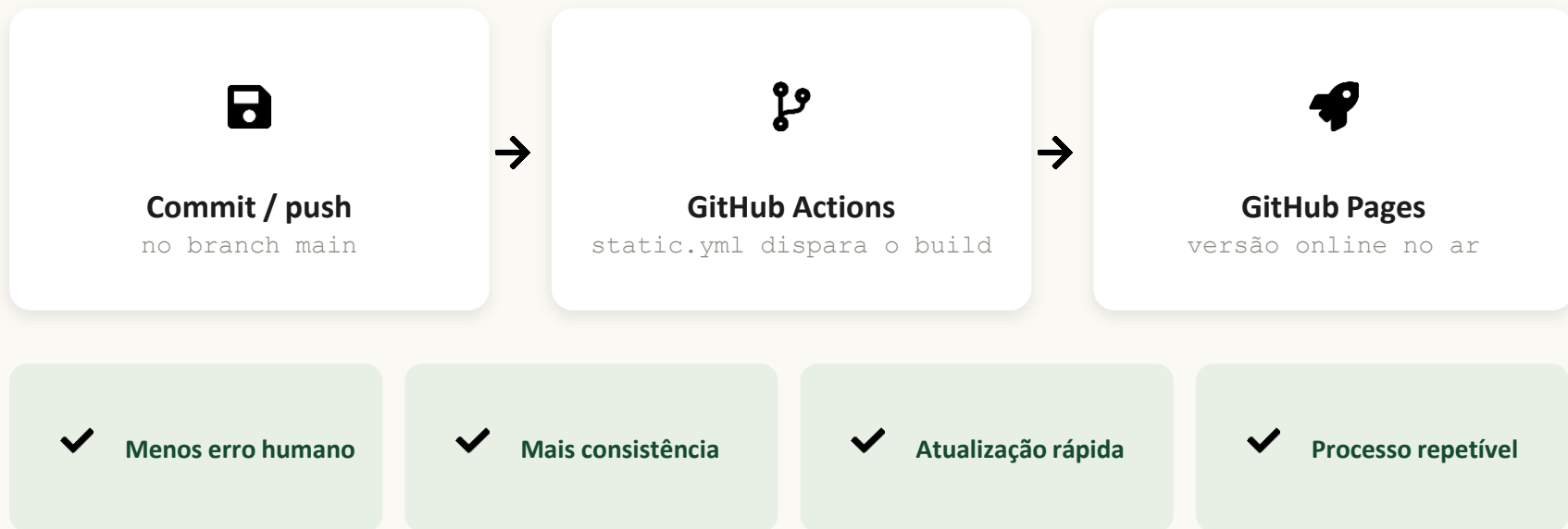
Testes automatizados `tests/`



Histórico preservado `versões paralelas (ifscee, ifscee001)`
`para consulta`

```
ifscee.org/  
├─ ifscee/  
├─ ifscee001/  
├─ src/  
├─ documentation/  
├─ tests/  
├─ .github/workflows/  
└─ index.html
```

Publicação automatizada



IA como copiloto, não como muleta.



Claude Code Pro, Gemini Pro e ChatGPT apoiam o desenvolvimento — sem terceirizar o raciocínio.



Esclarecer dúvidas



Feedback de sintaxe



Sugerir caminhos lógicos



Acelerar o aprendizado



Validar ideias



Estimular análise crítica

O objetivo foi aumentar a autonomia técnica — entender o problema, questionar a solução e validar o código.

RESULTADOS

O projeto em números.

13

capítulos
de manual

27

exemplos
de código

4

tutoriais
guiados

3

segmentos
de memória

100%

client-side
no navegador

0

instalações
necessárias

Desafios, aprendizados e futuro.



Desafios & aprendizados

- Simular memória fiel ao C real
- Implementar retrocesso de estado
- Sincronizar quatro painéis
- Modelar a AST de forma navegável



Trabalhos futuros

- Ampliar o subset de C99
- Mais exemplos e exercícios
- Modo professor / sala de aula
- Compartilhar sessões por link



IFSCee

Obrigado.

<https://www.ifscee.org>

IFSCee — *Código C que você pode ver.*

IFSCee — *C code you can see.*



github.com/gelasioebel/ifscee.org



gelasioebel.github.io/ifscee.org

Gelasio Ebel Junior

Projeto Integrador II · IFSC · ADS